

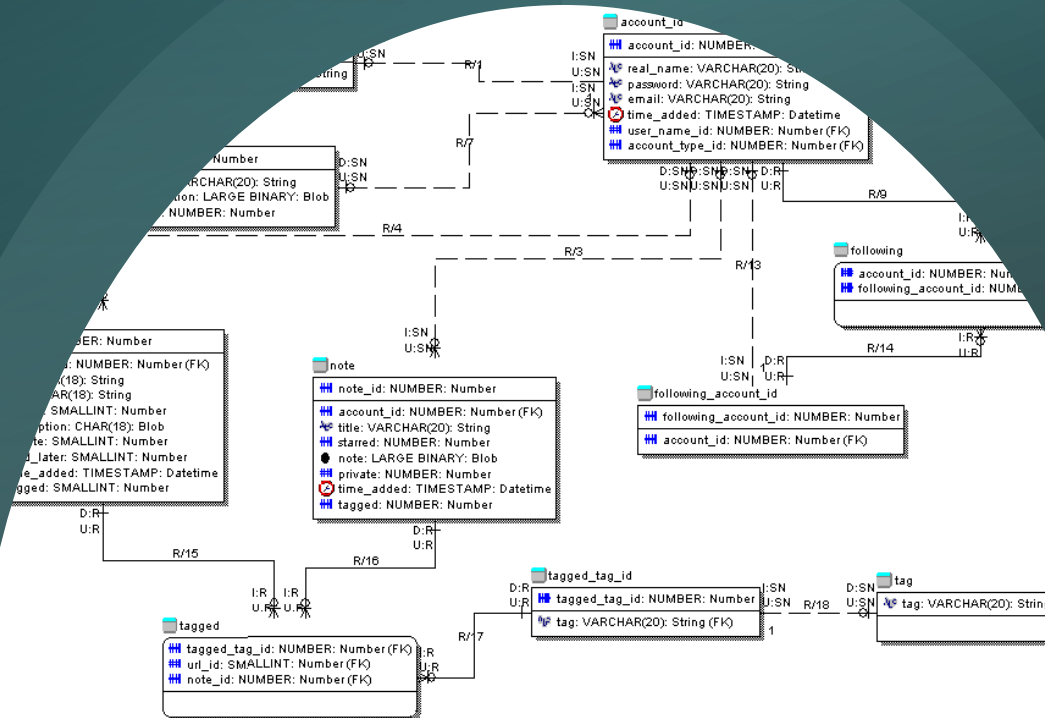
CS2300: File Structures and Introduction to Database Systems

Lecture 6: Relational Database Design by ER- and EER-to-Relational Mapping

San Yeung

Data Model Mapping

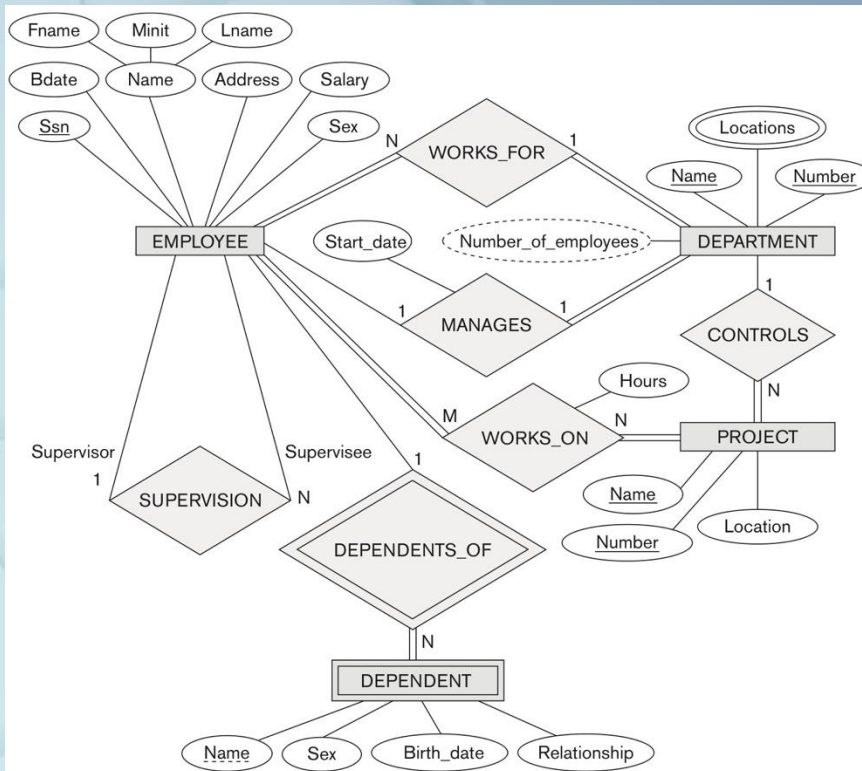
How to design a *relational database schema* based on a conceptual schema?



GOALS during Mapping

- Preserve **all information** (i.e. attributes)
- Maintain the constraints to the extent possible
 - *not all constraints can be preserved or directly mapped*
 - e.g. max cardinality ratio such as 1:10 in ER
- Minimize null values

ER-to-Relational Mapping Algorithm



Step 1: Mapping of Regular Entity Types

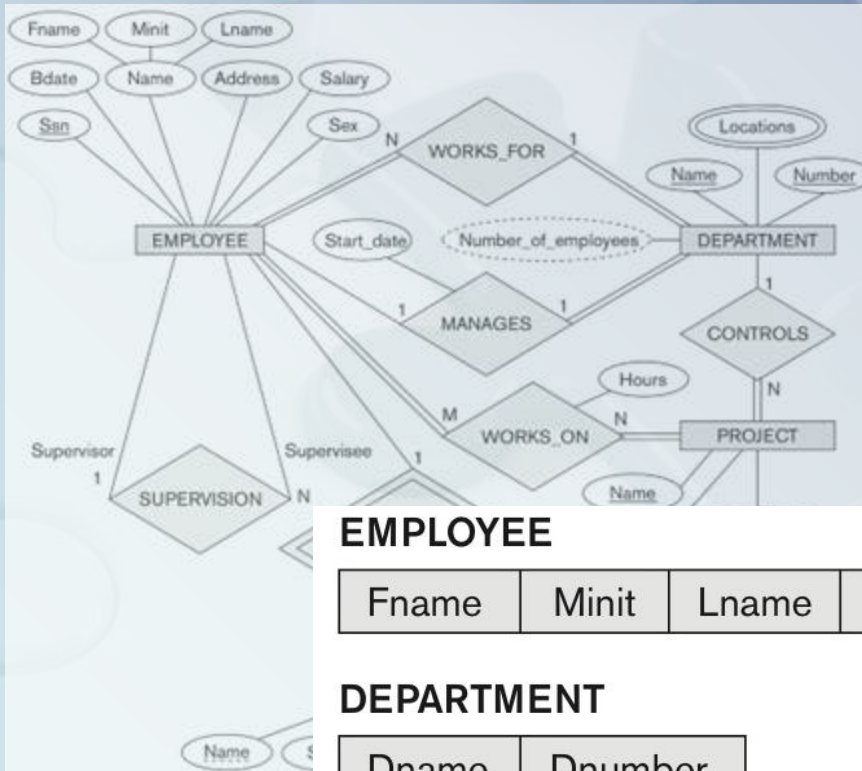
- Each *strong* entity type becomes a relation
- The relation should include all the **simple** attributes
 - what about the **composite** ones?
- Choose one of the candidate keys as the **primary key** for the relation

ER-to-Relational Mapping Algorithm

+

•

0



EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

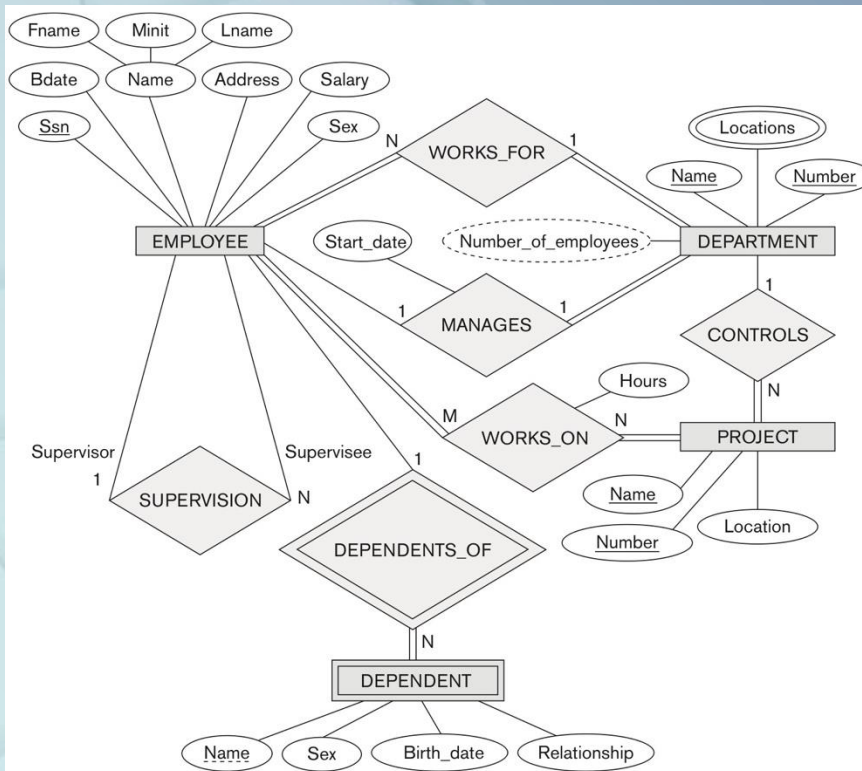
DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

ER-to-Relational Mapping Algorithm



Step 2: Mapping of Weak Entity Types

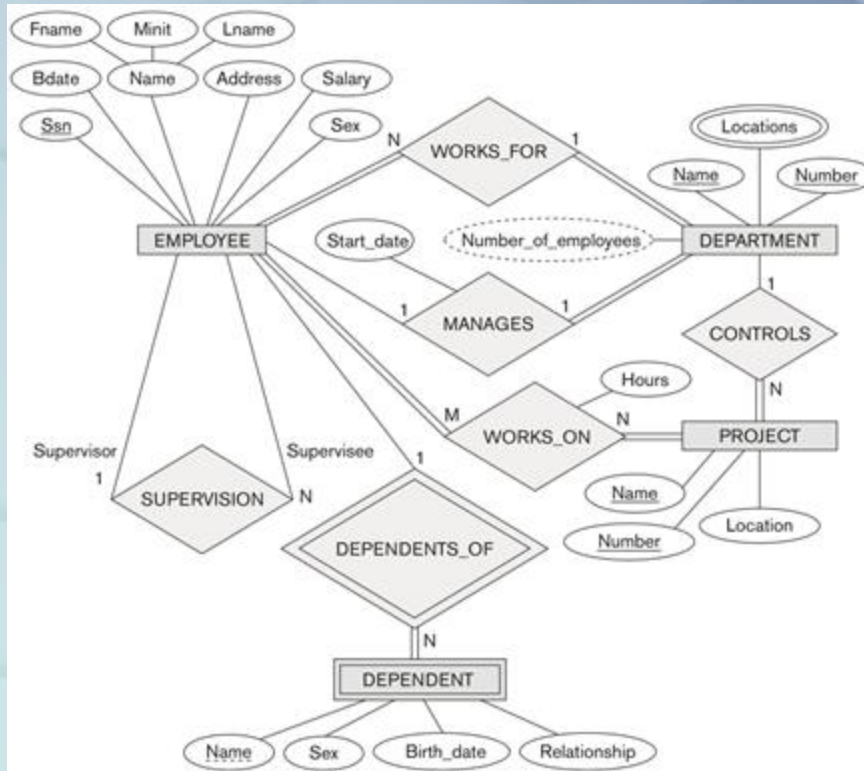
- Similar to that of strong entity types
- Addition: include the primary keys of its owners as **foreign keys** in the newly created relation
 - What would be the *primary key* for this relation?
- It is common to choose the propagate (**CASCADE**) option on the foreign key of such relation for the UPDATE and DELETE operations
 - Why?

ER-to-Relational Mapping Algorithm

+

•

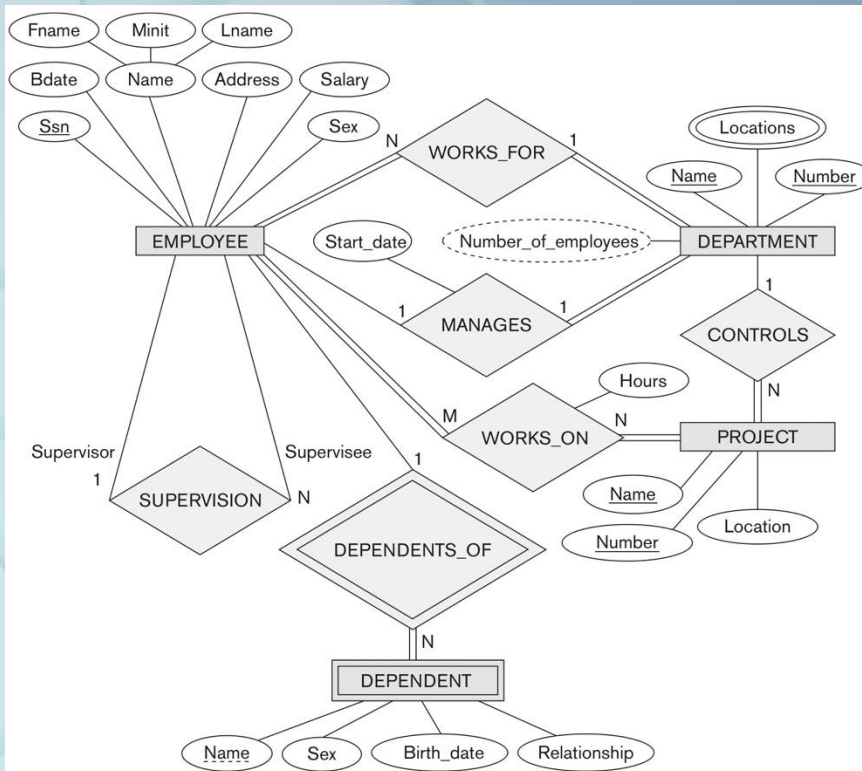
0



DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

ER-to-Relational Mapping Algorithm



Step 3: Mapping of Multivalued Attributes

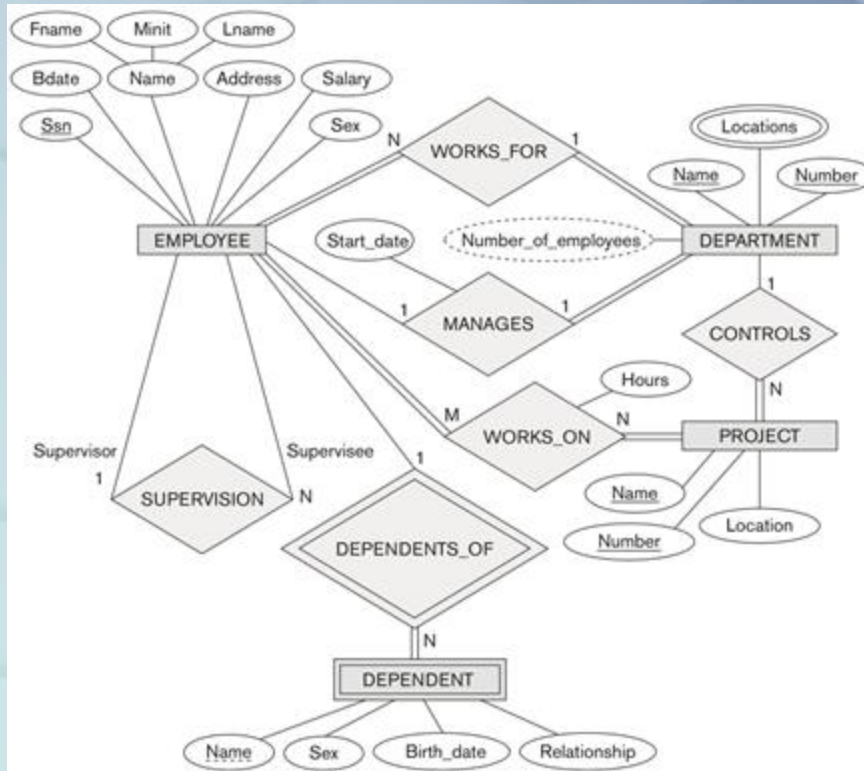
- Create a new relation for each multivalued attribute of an entity type E
- Include the primary key of E as a **foreign key** in the newly created relation
 - What would be the *primary key* for this relation?
- It is common to choose the propagate (**CASCADE**) option on the foreign key of such relation for the UPDATE and DELETE operations
- Note: a separate relation table is not required if the RDBMS supports *array data types*

ER-to-Relational Mapping Algorithm

+

•

0

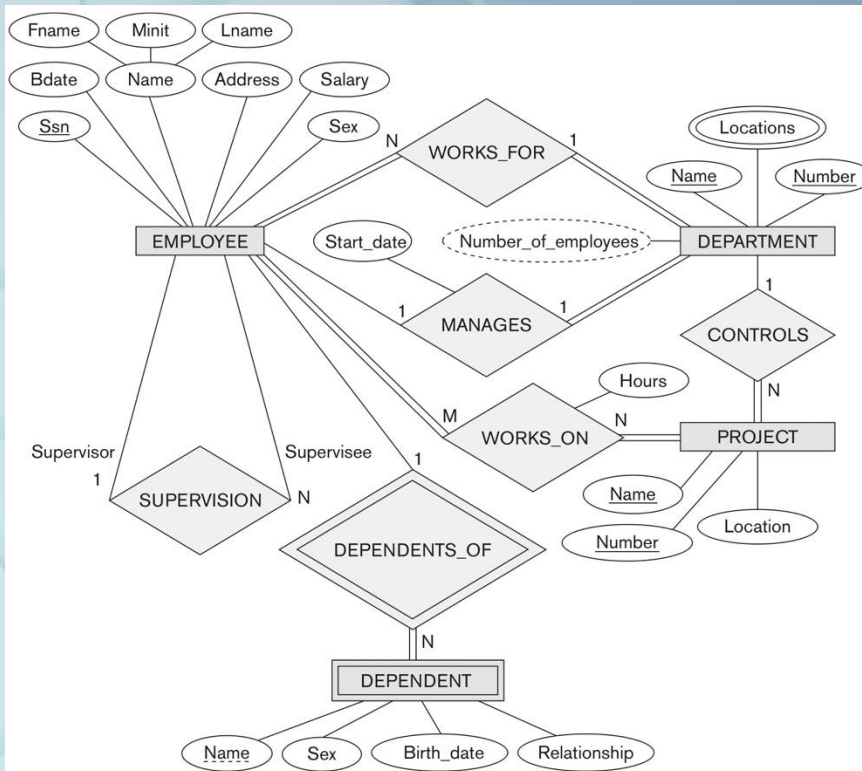


DEPT_LOCATIONS

Dnumber

Dlocation

ER-to-Relational Mapping Algorithm



Step 4: Mapping of Binary 1:1 Relationship Types

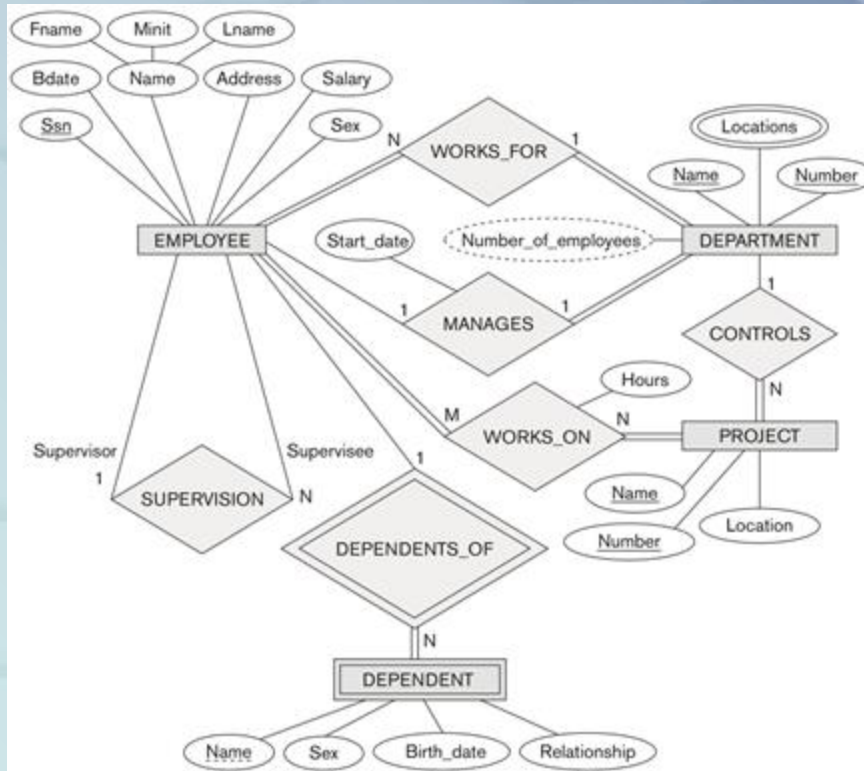
- The **Foreign Key** approach
- Choose one of the relations (S) to include a new **foreign key** based on the **primary key** of the other relation (T)
- It is better to choose an entity type with *total participation* in the role of S
 - **Why?**

ER-to-Relational Mapping Algorithm

+

•

0



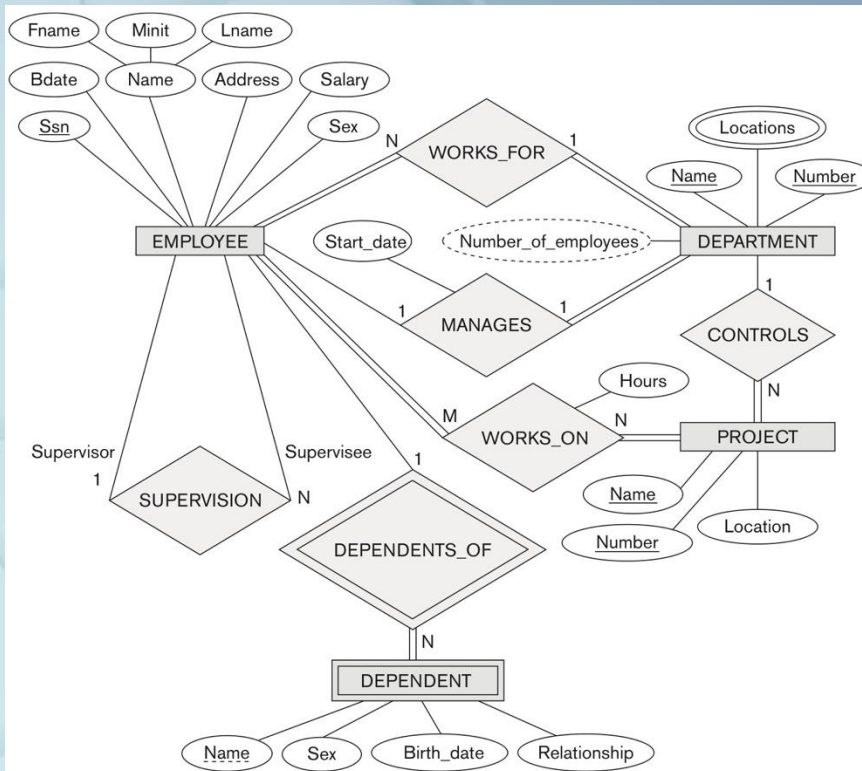
Dname

Dnumber

Mgr_ssn

Mgr_start_date

ER-to-Relational Mapping Algorithm



Step 5: Mapping of Binary 1:N Relationship Types

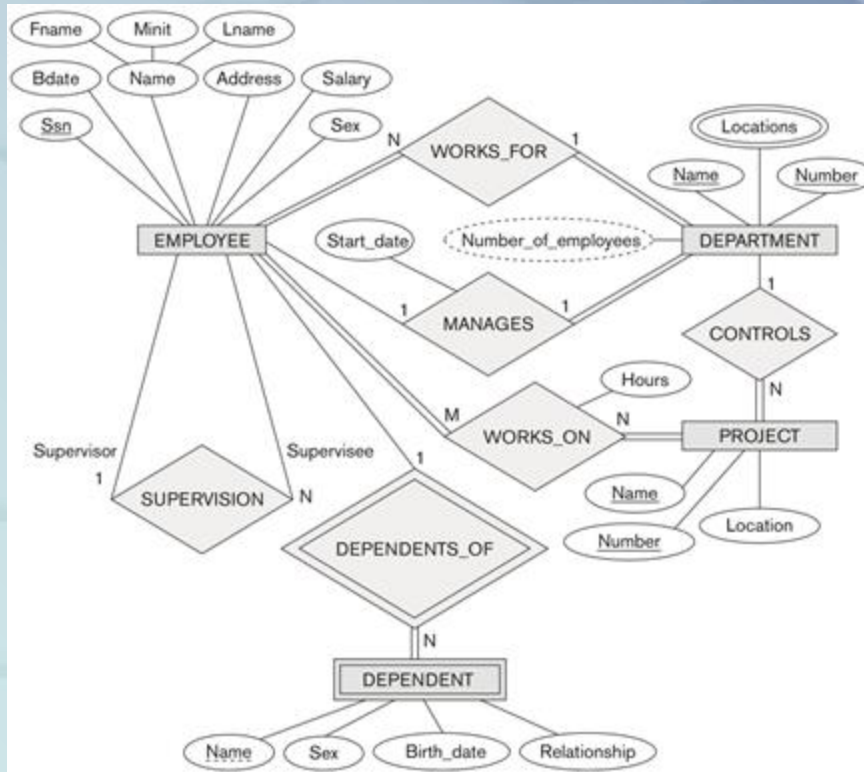
- The **Foreign Key** approach is preferred as it reduces the number of tables
- Choose one of the relations (*S*) to include a new **foreign key** based on the **primary key** of the other relation (*T*)
- It is better to choose an entity type participating at the *N-side* of the relationship type as *S*
 - **Why?**

ER-to-Relational Mapping Algorithm

+

•

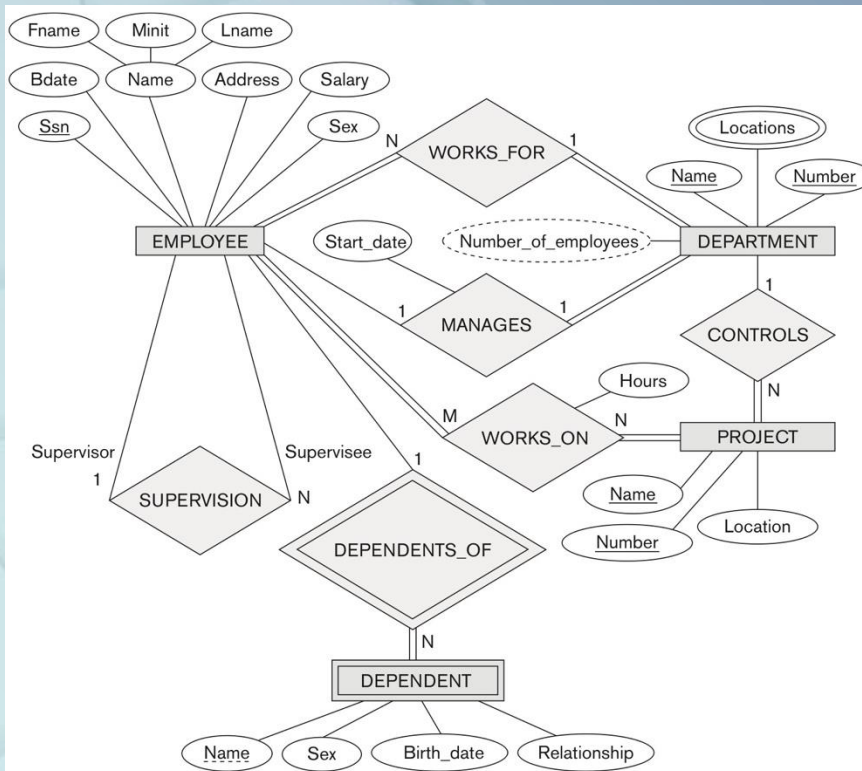
0



Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

ER-to-Relational Mapping Algorithm

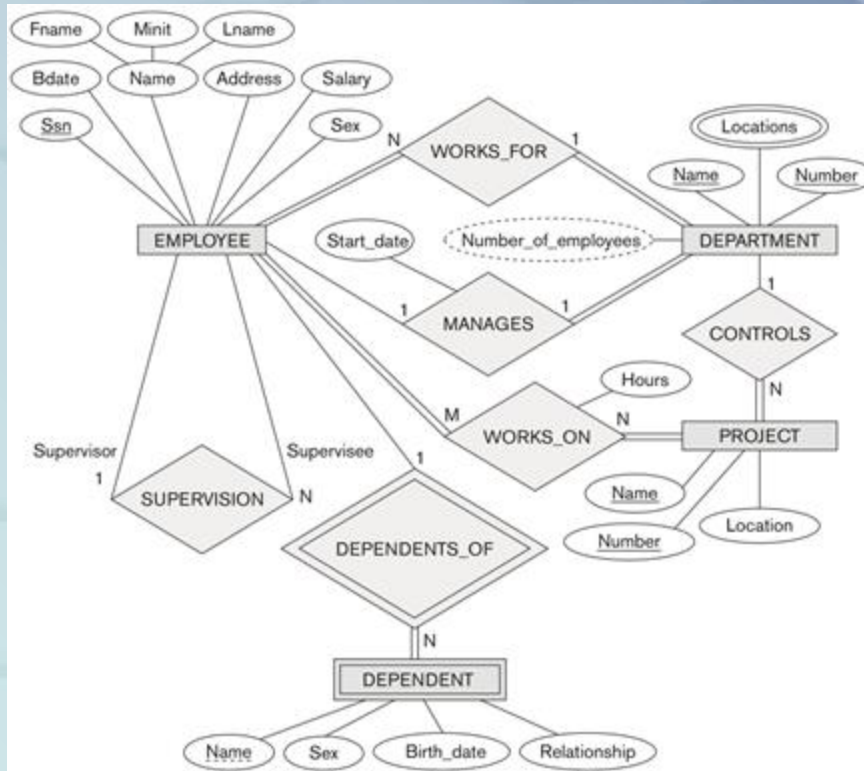


Step 6: Mapping of Binary M:N Relationship Types

- The **Relationship Relation (Cross-Reference)** approach
- Create a new relation to represent the relationship of the participating entity types
 - What would be the **primary key** for this relation?
 - What would be the **foreign keys** for this relation?
- This approach also works for the mapping of **N-ary relationship types**

ER-to-Relational Mapping Algorithm

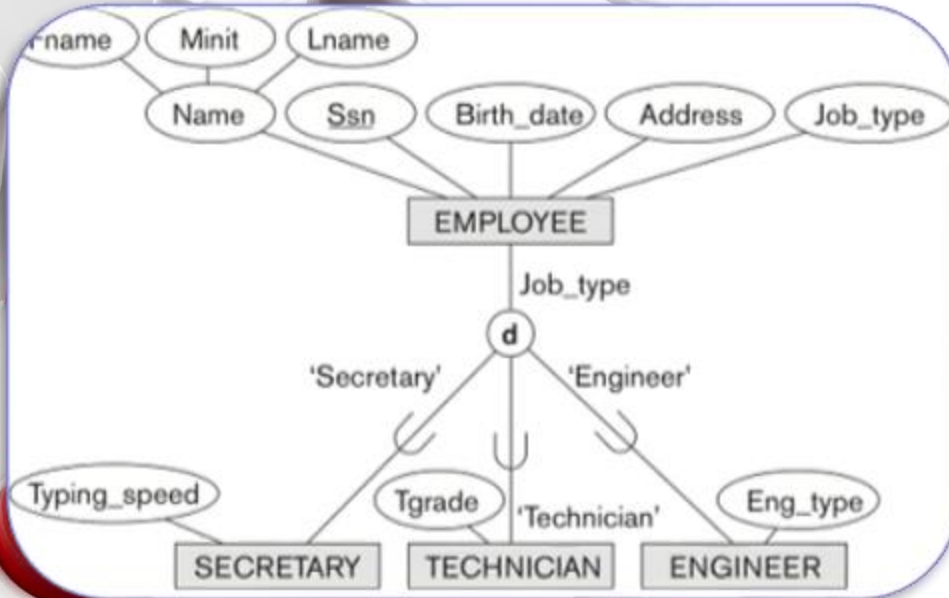
+
•
0



WORKS_ON

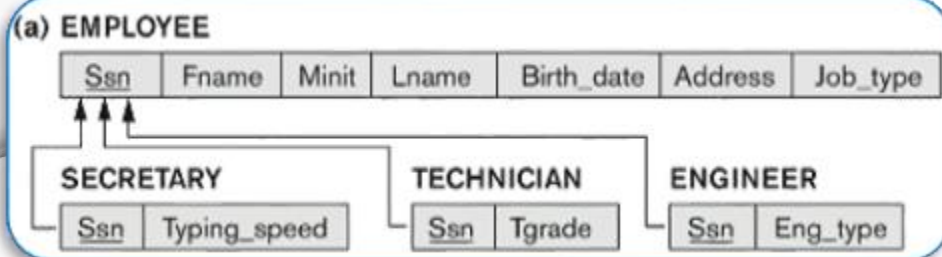
<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

EER-to-Relational Mapping Algorithm

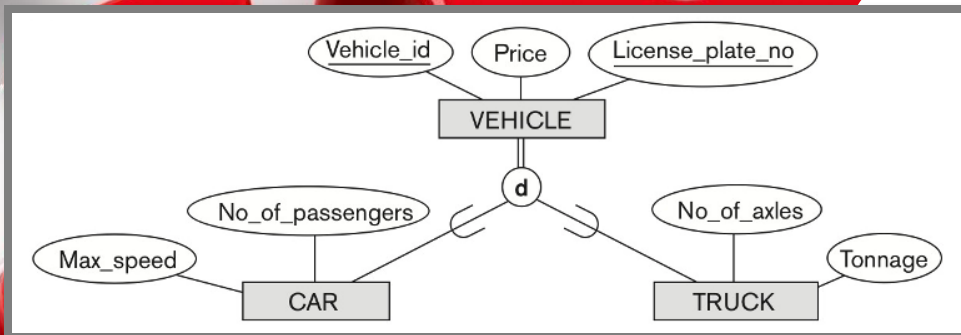


Step 7: Mapping of Specialization or Generalization

- The two main options are to map the specialization/generalization into a *single* or *multiple* tables
- **Option A: Multiple relations – superclass and subclass**
 - Create a relation for the superclass plus a relation for each subclass
 - Works for any constraints on the specialization: *disjoint* or *overlapping*, *total* or *partial*



EER-to-Relational Mapping Algorithm



Step 7: Mapping of Specialization or Generalization

- The two main options are to map the specialization/generalization into a *single* or *multiple* tables
- **Option B: Multiple relations – subclass relations only**
 - The superclass is **integrated** into the subclass relations
 - This option works well when both the _____ and _____ constraints hold

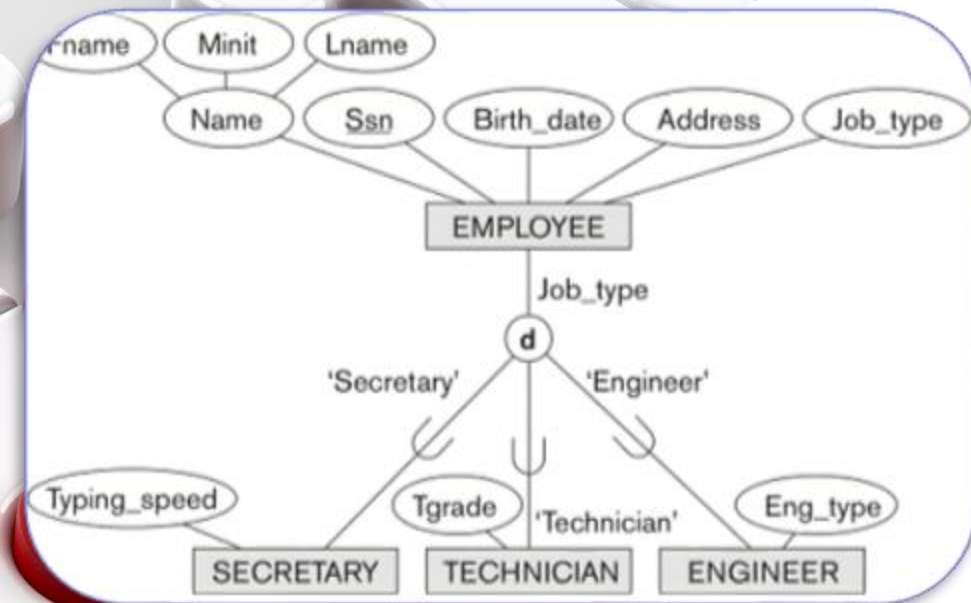
(b) CAR

<u>Vehicle_id</u>	License_plate_no	Price	Max_speed	No_of_passengers
-------------------	------------------	-------	-----------	------------------

TRUCK

<u>Vehicle_id</u>	License_plate_no	Price	No_of_axles	Tonnage
-------------------	------------------	-------	-------------	---------

EER-to-Relational Mapping Algorithm



Step 7: Mapping of Specialization or Generalization

- The two main options are to map the specialization/generalization into a *single* or *multiple* tables
- **Option C: Single relation with one type attribute**
 - Create a single relation to represent the superclass and all its subclasses
 - This option is used to handle *disjoint* subclasses by including a single **type attribute** *t* to indicate which subclass each tuple belongs
 - *Should we worry about this having too many NULL values?*

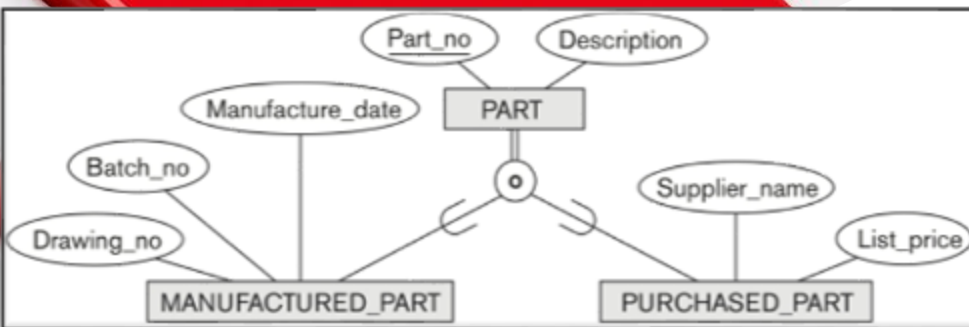
(c) EMPLOYEE

<u>Ssn</u>	Fname	Minit	Lname	Birth_date	Address	Job_type	Typing_speed	Tgrade	Eng_type
------------	-------	-------	-------	------------	---------	----------	--------------	--------	----------

EER-to-Relational Mapping Algorithm

Step 7: Mapping of Specialization or Generalization

- The two main options are to map the specialization/generalization into a *single* or *multiple* tables
- **Option D: Single relation with multiple type attribute**
 - Create a single relation to represent the superclass and all its subclasses
 - This option is used to handle _____ subclasses by including multiple **Boolean** type fields to indicate which subclasses each tuple belongs
 - *Would this option work for a disjoint specialization as well (technically)?*



(d) PART

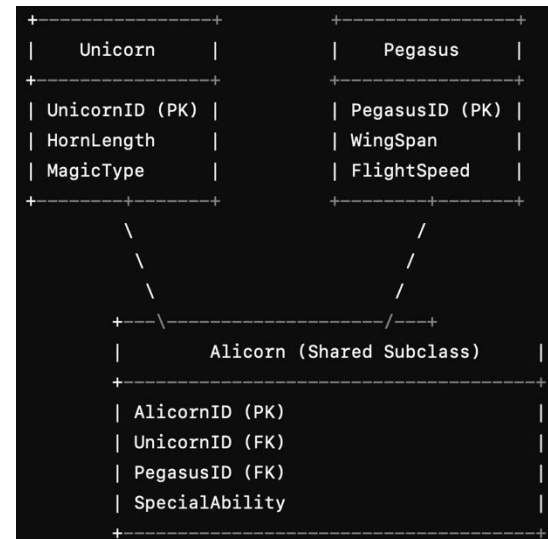
<u>Part_no</u>	Description	Mflag	Drawing_no	Manufacture_date	Batch_no	Pflag	Supplier_name	List_price
----------------	-------------	-------	------------	------------------	----------	-------	---------------	------------



EER-to-Relational Mapping Algorithm

Step 8: Mapping of Shared Subclasses (Multiple Inheritance)

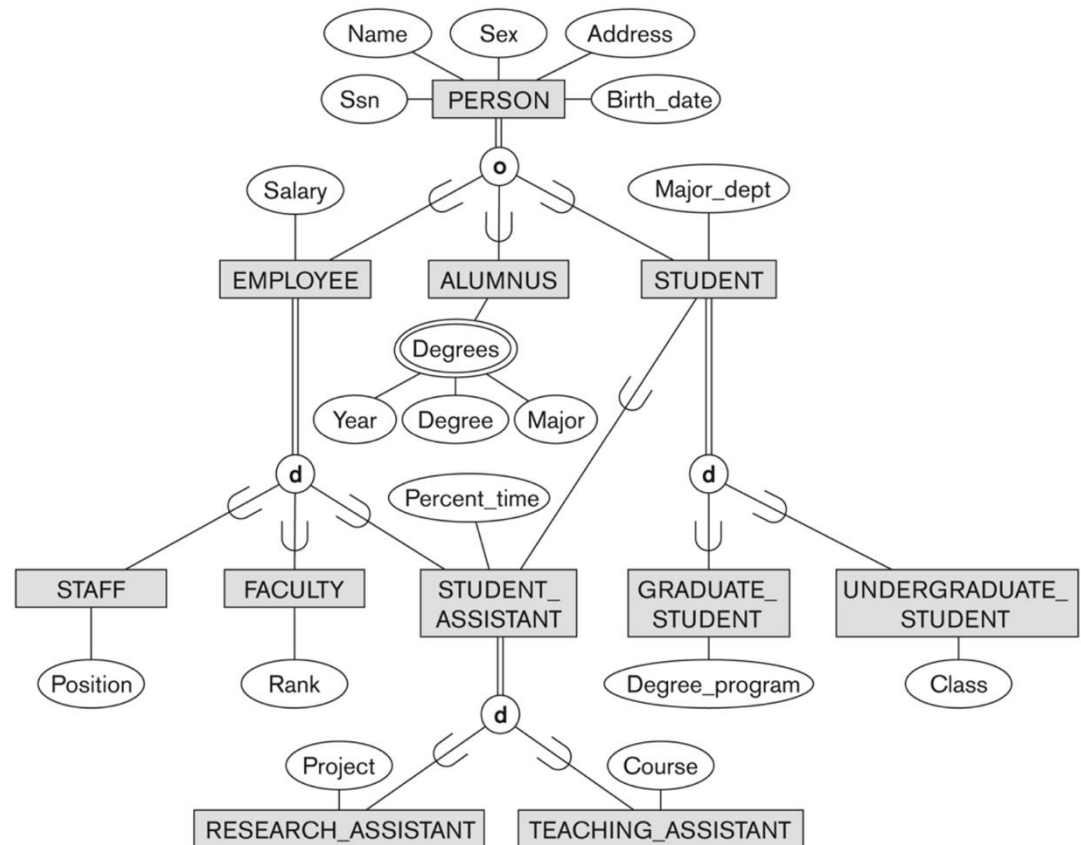
- A **shared subclass**, such as ENGINEERING_MANAGER, is a subclass of several superclasses (that shared the same key attributes)
- Any of the four options presented is applicable to shared subclasses



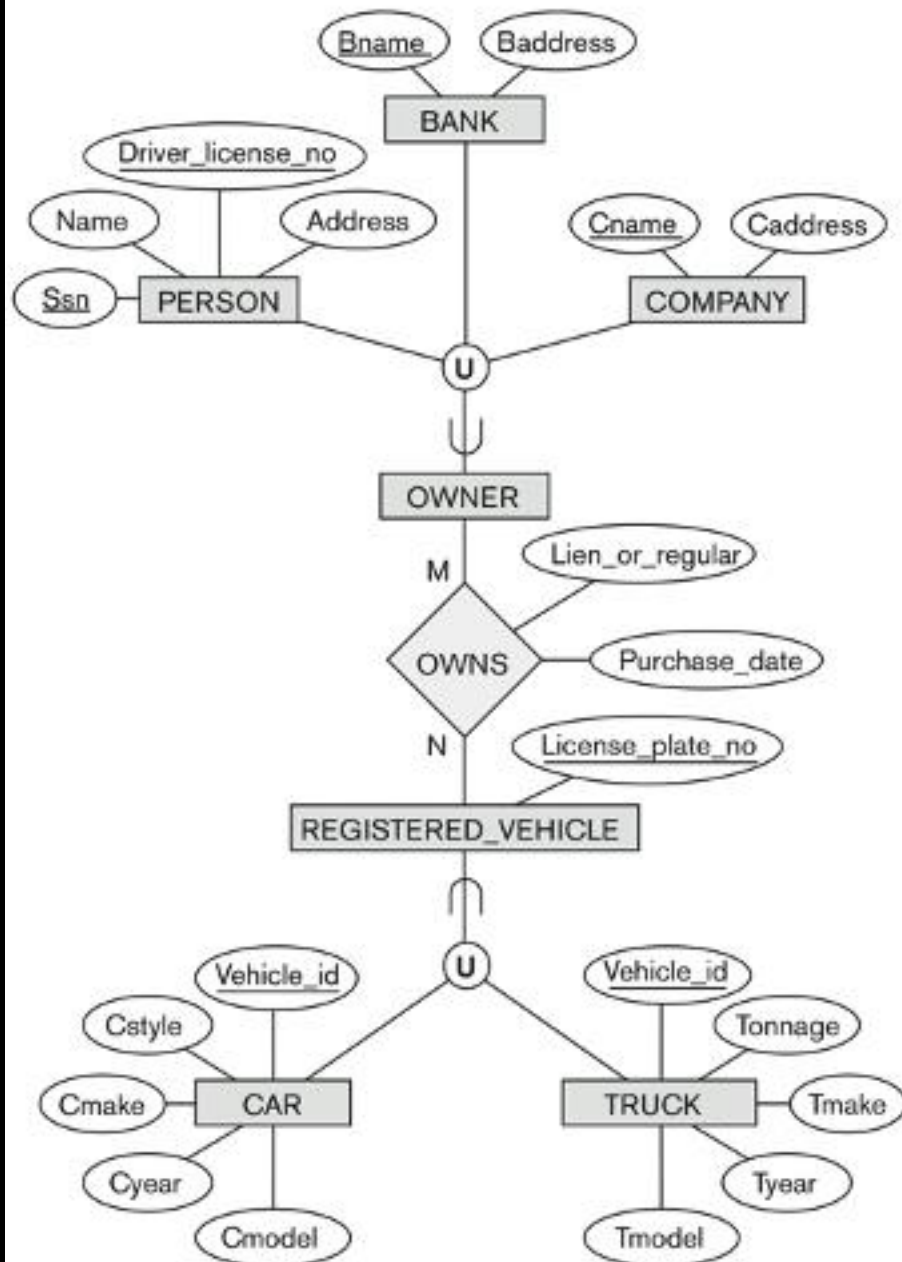
Mapping of Shared Subclasses

• *Exercise:*

- First, map PERSON, EMPLOYEE, ALUMNUS, and STUDENT. [+1 B.P.]
- Then, map STUDENT_ASSISTANT. [+1 B.P.]



EER-to-Relational Mapping Algorithm



Step 9: Mapping of Categories (Union Types)

- A category is a subclass of the *union* of two or more superclasses that might have different keys.
- Accommodate union types by specifying a **surrogate key** if the keys of the defining classes are different.

ER- and EER-to-Relational Mapping Summary

Table 9.1 Correspondence between ER and Relational Models

ER MODEL	RELATIONAL MODEL
Entity type	<i>Entity</i> relation
1:1 or 1:N relationship type	Foreign key (or <i>relationship</i> relation)
M:N relationship type	<i>Relationship</i> relation and <i>two</i> foreign keys
<i>n</i> -ary relationship type	<i>Relationship</i> relation and <i>n</i> foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key